

EFCC: Ethernet Frame Crafter & Capture for TSN Research

Akram BEN AHMED, Takahiro HIROFUCHI, Takaaki FUKAI
Intelligent Platforms Research Institute
National Institute of Advanced Industrial Sciences and Technology
2-3-26 Aomi, Koto-ku, Tokyo, JAPAN
{akram.benahmed,t.hirofuchi,takaaki.fukai}@aist.go.jp

Abstract—Time-Sensitive Network (TSN) has been considered one of the most viable solutions to meet the increasing demands for ultra-low latency in the 5G and post-5G eras. Due to the importance of timing constraints, network measurement tools are necessary for TSN research and operations to evaluate the performance limits of proofs-of-concept, troubleshoot failures, etc. However, current measurement solutions struggle to achieve important aspects of TSN networks, such as precise control of the transmission frame intervals and burst sizes. Additionally, they cannot generate a mix of multiple flows of various characteristics. In this paper, we present Ethernet Frame Crafter & Capture (EFCC), an FPGA-based network measurement tool to address the aforementioned issues. The proposed system allows the frame generation/capture functions to be configured independently for each port at a line rate of 1GbE. Furthermore, EFCC is open-source and can be ported to most commodity FPGA boards available in the market. The evaluation results demonstrate that the proposed frame generator allows the mixing of multiple flows of various characteristics and that the frame capture module can record the transmit and receive timestamps of frames with a precision of 8-ns/6.4-ns.

Index Terms—Time Sensitive Networks, Frame Generator, Frame Capture, FPGA

I. INTRODUCTION

Time Sensitive Networks (TSN) [1] have recently seen widespread adoption to respond to the increasing demands in time-sensitive traffic with ultra-low latency of milliseconds. These timing requirements have become more important with the emergence of new technological paradigms such as the Internet of Things, Edge computing, and 5G communication. To satisfy these timing requirements, TSN introduces additional amendments to conventional IEEE 802.1 and 802.3 Ethernet network standards aimed at providing deterministic guarantees, robustness, and integrated diagnostics and management services [2].

Traffic measurement tools play a critical role in network design, research, and operation by ensuring accurate performance, reliability, and efficiency. They enable the generation and capture of test frames in advance, facilitating the assessment of network performance. In the context of TSN, this becomes even more vital due to the added requirements for determinism, ultra-low latency, and accuracy. Not forget to mention that TSN measurement tools should support multiple flows with different characteristics as they typically coexist in the same network.

Despite the importance of these requirements, none of the TSN measurement tools proposed so far is capable of properly satisfying them. Most of them are based on software simulations, whose accuracy is always questioned due to the inevitable gap between simulation and real-world behaviors. Other solutions use Commercial Off-The-Shelf (COTS) reconfigurable hardware switches or custom processors for traffic generation and capture. Although the accuracy can be improved, they are unable to properly generate the desired mixture of flows and characteristics that is required by TSN. Recently, TSN research has been steadily shifting from theoretical modelization/simulation towards more real working systems where practical TSN systems must accurately transfer a mixture of flows having different frame sizes, frame intervals, and burst sizes. Therefore, the need for a fully customizable and reconfigurable network measurement environment has never been so crucial for both academic and industrial research and development.

Starting from the fact mentioned above, we propose in this paper Ethernet Frame Crafter & Capture (EFCC), a flexible FPGA-based frame generation and capture system targeted for TSN research and development. Thanks to the reconfigurability and flexibility of FPGAs, the proposed open-source system [3] is capable of generating multiple TSN flows with different characteristics, where the frame size, frame rate, and burst size are independently set for each flow. In addition, it can record the arrival times of all the frames with a high-precision hardware clock without any loss of the arrival time records, even with the shortest frame size. The proposed system is designed on affordable FPGAs and offers TSN researchers a high degree of flexibility and the opportunity to evaluate TSN networks under different traffic scenarios and different mixtures of flows. To the best of our knowledge, EFCC is the first attempt in the literature to present a hardware-based, open-source, and configurable measurement tool for TSN research.

The rest of this paper is organized as follows: Section II discusses some of the related works on TSN testing and evaluation. In Section III, we present our FPGA-based EFCC system in a fair amount of detail. Section IV is dedicated to the preliminary evaluation, validation, and analysis of our proposed implementation. Finally, we conclude the paper with summarizing remarks in Section V.

II. RELATED WORK

Network measurement has been a well-established discipline in conventional Ethernet networks. When it comes to TSN research, simulation frameworks are the most widely used. Notably, OMNeT++ has been extensively used to simulate TSN networks and explore its different features [2], [4]–[6]. They are useful for studying theoretical aspects of TSN algorithms, but not for evaluating real-world TSN systems. For experiments in the real world, measurement programs designed for normal networks (e.g., Iperf and MoonGen [7]) are not suitable for TSN studies that require comprehensive latency measurements.

In the realm of hardware-based solutions, few industrial TSN network testers have been developed. However, detailed implementation specifics are not openly provided. Academia has also seen relatively few studies focusing on TSN hardware methodologies. For instance, *Li et al.* [8] proposed a network I/O mechanism to achieve microsecond-level latency, named Direct Register Access (DRA). DRA tries to mitigate the latency caused by the data movement between internal registers and external storage. Similarly, they later proposed the DetChip [9], a system-on-chip capable of deterministically implementing the TSN scheduling results. When these techniques are used for implementing measurement mechanisms for the performance of TSN, they may offer better accuracy and more reliable results, compared to ordinary network measurement programs running on COTS computers. However, their motivation is to implement time-sensitive endpoint systems, which does not discuss the design of the measurement mechanism we aim for.

A study in [7] discussed the possibility of using a Tofino-based frame generator [10] to reproduce frame intervals that can be observed in TSN systems, rather than measuring the performance of the TSN system itself.

III. PROPOSED EFCC DESIGN

The proposed EFCC system supports the simultaneous generation and capture of TSN frames from multiple 1 Gbps physical ports. This can be easily done by configuring which function of the frame generator/capture is used for each port, thanks to the reconfigurability properties of FPGAs. It can function autonomously in standalone mode or be attached to a host machine when much larger frame-generation/capture buffers are needed.

The frame generator module (FG) is capable of creating multiple flows with different characteristics and can independently set the frame size, frame rate, and burst size for each flow. It can mix different frame sizes within a single flow. If the timestamp recorder module (TR) for transmission is enabled, it is capable of recording the transmission times of all the frames with a high-precision hardware clock. For frame capture, the receiving timestamp recorder can record the arrival times of all the frames with the same high-precision hardware clock as the frame generator, without any loss of arrival time records, even with the shortest frame size. EFCC can be implemented with commodity FPGA boards, which are

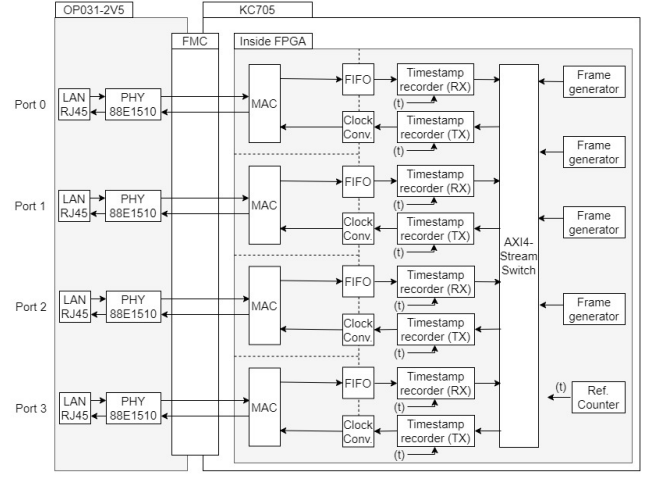


Fig. 1: Configuration example of the proposed EFCC system

much cheaper than dedicated test appliances, and can be easily ported to any other FPGA board with minimal modification to the design. Hereafter, we present the key architectural and functional details of our system.

A. Overall Architecture

As shown in Figure 1, the proposed design implements up to four FGs and eight in-line TRs, allowing the latency measurement of frames output from the frame generators. Both FG and TR blocks support 1000BASE-T with a maximum designed speed of 1 Gbps and two Ethernet frame types: Ethernet II and IEEE 802.3ac (VLAN tag support). In addition, we set the Maximum Transmission Unit (MTU) to be 1500 bytes. It is worth mentioning that this figure depicts a simple configuration example, and users can easily change the number of FGs and TRs, connect any input port (frame generator or MAC RX) to any output port (MAC TX), and measure the latency of any connection.

As illustrated in Figure 1, the main components of EFCC are implemented on an AMD Kintex 7 FPGA KC705 Evaluation Kit [11]. Since KC705 has only a single Ethernet port, we attached to it an Opsero OP031-1V8 Ethernet FMC [12] via the “FMC HPC” connector, as can be seen in Figure 1. This card is an add-on/expansion board that hosts four Marvell 88E151x Gigabit Ethernet PHYs to provide four ports of Gigabit Ethernet connectivity to the KC705 board.

Incoming streams from an Ethernet port are initially directed to the MAC block. It facilitates data bridging between the PHY (utilizing the RGMII interface) and the FPGA programmable logic section via an AXI4-Stream bus. Within the MAC block, incoming streams undergo processing where the Preamble, Start Frame Delimiter (SFD), and Frame Check Sequence (FCS) fields are removed from the input Ethernet frames. Conversely, these fields are added for outgoing frames. Later, frames are stored in the input FIFO blocks, as shown in Figure 1. These FIFO blocks evidently serve for storing purposes, as well as for clock conversion between the MAC

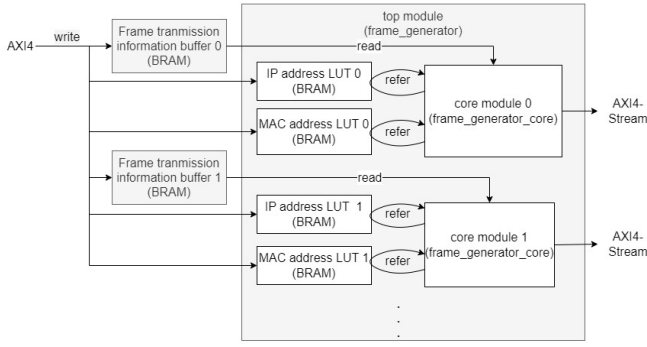


Fig. 2: Frame generator block diagram

side and the FPGA core side. Both domains work at a 125MHz clock frequency. However, they have different clock sources; therefore, synchronization is needed.

B. Frame Generator block

The frame generator module (FG) generates and transmits frames based on the transmitted frame information written to BRAM in advance. The accommodated frame size is from 64 bytes to 1518 bytes, where if a VLAN TAG is assigned, the maximum frame size is 1522 bytes. In this module, UDP or RAW IPv4 frames are supported. To provide an accurate frame interval that is necessary for TSN frame transmission, the interval between frames to be transmitted can be adjusted in 1-cycle units, with a minimum of 20 cycles @125MHz (i.e., the duration of Preamble, SFD, and Interframe Gap).

As can be seen in Figure 2, the top module is configurable and can contain as many “core modules” as the number of physical output ports. This offers users the ability to define how many FGs can start at the same time. Each “core module” refers to two fixed-size BRAM blocks each containing the table of the IP addresses and that of MAC addresses, which are referred by the frame sequence information, respectively. Each “core module” is connected to a “Frame transmission information buffer” which stores the information of the frame sequence to be transmitted. The size of this information can be configured by the users; thus, offering freedom on the type and amount of data that needs to be transmitted.

The “core module” loads the 16-byte entry for each frame transmission from the “Frame transmission information buffer” to generate and transmit frames. These entries contain 3 types:

- *Normal frames*: An entry to compose a frame sent to the outside world (i.e., latency measurement target)
- *NOP frames*: An entry of a dummy frame to adjust frame intervals, not for composing a frame sent to the outside world
- *EOL frame*: An entry of a dummy frame that indicates the end of a frame sequence

When the FG starts, it reads the frame information sequentially from the beginning of the BRAM and performs processing according to that information. When an EOL frame is reached, the FG stops. We define in our implementation the

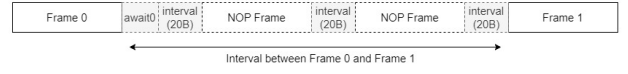


Fig. 3: Transmission rate control mechanism

Repeat Mode. When it is enabled and the FG starts, it does not stop after reaching an EOL frame. Instead, it continues processing from the beginning of the BRAM again.

The frame generator can insert any interval between frames by using NOP frame entries and the Await field of a normal frame entry, as illustrated in Figure 3. A NOP frame entry can specify the size of the dummy frame in its 16-byte payload size field. The minimum frame interval inserted by a NOP frame entry is 672 ns, which is equal to the duration to send a frame of the shortest size (i.e., 84 bytes at the physical layer). The maximum frame interval inserted by a NOP frame entry is 524840 ns. NOP frame entries are repeatable. To adjust frame intervals in a more fine-grained manner, the 7-bit Await field of a normal frame entry is used. When specified, the frame interval after the frame is sent is increased by the specified number of cycles up to 127 (i.e., 1016 ns).

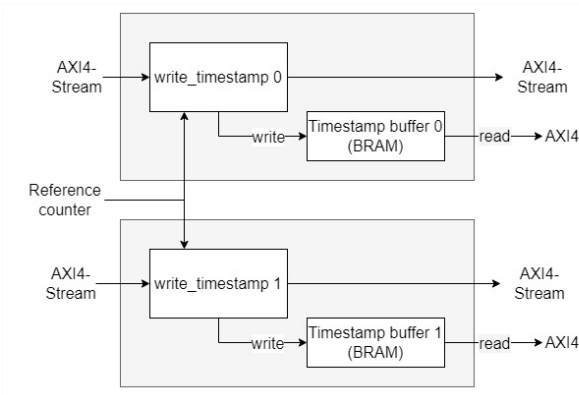
It is important to mention that the behavior of the FG can be controlled by users without the need to compile the design again. In other words, once the physical parameters are decided (e.g., number of physical ports, buffers’ sizes, etc.) and the design bitstream is generated and downloaded to the FPGA, users can easily configure the number of active frame generators and the traffic to be generated via simple commands sent through JTAG.

C. Timestamp Recorder block

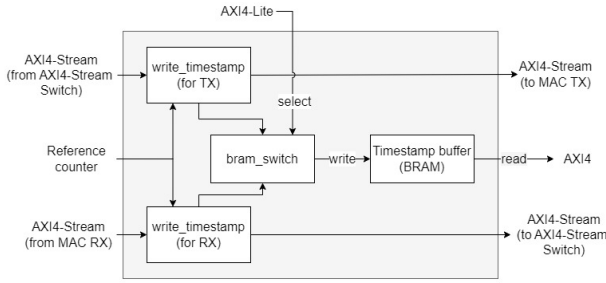
The timestamp recorder module (TR) relays the input Ethernet frames as they are with zero latency. In our design, the FG embeds a special data called *Magic Word* and the index number of each frame into the payload of the frame. The TR finds the frame sent from the FG, distinguishing them from other frames by using the *Magic Word*. It then saves the arrival time of each frame and its index number in BRAM. Therefore, the input protocols and frame structures that can use the timestamp recording function are limited to those that can be generated by the FG.

1) *Timestamp Recorder architecture*: The TR is composed of the “write_timestamp” and the “timestamp buffer” modules. The former extracts the counter value of the time at which the beginning of AXI4-Stream passed through. The extracted value counter is then stored in the Timestamp BRAM buffer. The counter value is read from the “Ref. Counter” module which generates and manages a reference unsigned integer 32-bit counter for each module, as represented in Figure 1.

As depicted in Figure 4, the TR can be used in either an independent BRAM configuration, where each “write_timestamp” uses one BRAM, or a shared BRAM configuration, where two “write_timestamp” modules share one BRAM. By making this distinction, we aim to provide



(a) Independent buffer configuration



(b) Shared buffer configuration

Fig. 4: Timestamp recorder block diagram

users with a degree of flexibility according to their requirements and the available resources. The independent BRAM configuration, depicted in Figure 4a, has a high degree of freedom in terms of the manipulation of the recorded data; but, consumes a lot of BRAM resources. On the other hand, in the shared configuration, represented in Figure 4b, only one of the “write_timestamp” modules can write to the BRAM; however, this reduces BRAM resources.

In the case of the shared BRAM configuration, a “BRAM Switch” component is added to the design, as shown in Figure 5. It selects one of the two “write_timestamp” modules to access the BRAM. The output signal can be selected from the AXI4-Lite port and it is possible to set that neither module can write to the BRAM. Moreover, one-stage pipeline registers can be added to the input and output ports by IP settings.

2) *Timestamp recording procedure:* By placing the TR at the point where the latency is to be measured, a timestamp is automatically recorded in BRAM. The TR itself has zero latency and works transparently to other modules as it relays input data as it is without consuming any clock cycles. Thus, it does not affect the behavior of the design. As previously stated, when a frame containing the *Magic Word* is received, the ID contained in the frame and the timestamp of the time when the beginning of the frame passed are saved to the BRAM. The ID contains the port number that identifies the frame generator from which the output originated, as well as the frame number that indicates which frame the frame generator created. The

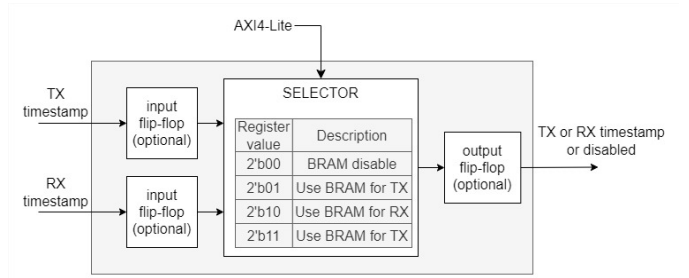


Fig. 5: BRAM switch architecture

timestamp saving process is performed until the BRAM free space is exhausted. To save the timestamp after the BRAM free space is exhausted, users must perform a reset process.

3) *Latency correction:* The transmitted frame within EFCC is affected by a certain latency caused by the different components of the system. It is important to correct this latency for accurate calculation based on the recorded timestamps. This design step is important as, without it, the recorded timestamp will be erroneous and will not correctly depict the timing behavior of a given traffic. In the case where latency is measured from the FPGA design to the outside and back, the latency that occurs inside the FPGA design must be subtracted. By finding the difference between the latency when connected to the device to be measured and the latency when directly connected with an Ethernet cable, the latency generated in the FPGA design, which is the common part, is excluded, and the latency from input to output of the desired device can be obtained.

For a flexible design, we made sure that the correction delay could be configured at run time by modifying a register in the TR via simple JTAG commands. In other words, a timestamp with the latency already corrected can be recorded by setting the correction value in the configuration register in advance. The timestamp is a counter value that is driven by a 125 MHz clock, so a correction value of 1 corresponds to 8 ns.

IV. EVALUATION

A. Latency measurement

First, we conducted experiments to validate the accuracy of our latency measurement method. For validation, one could use a commercial product that claims to support latency measurement and compare our results with theirs. However, the products we tested did not output stable values that are consistent with their datasheets, possibly due to an issue in synchronization between sender and receiver appliances. Since there is no certificate for the accuracy of the latency measurement capability in such products, we consider that it is not currently suitable for us to use another system as a reference to validate our measurement system. Instead, we opted to measure the latencies of different lengths of Ethernet cables and compared them with theoretically expected latencies. The signal propagation speed of a copper Ethernet cable is approximately estimated from the Nominal Velocity

of Propagation (NVP) in its datasheet, which is generally around 65% to 70% of the speed of light in a vacuum. If our measurement system is accurate, it is expected that the latencies of Ethernet cables will match the theoretical estimates.

The measurement system adjusts the recorded transmit and receive timestamps so that they refer to the moments of transmission and reception at the Ethernet ports. By looping back frames via a short cable, we measured the internal latency of the measurement system, which does not depend on the device under test. It was approximately 100 cycles (i.e., 800 ns). This value was used as the calibration value set in the configuration registers, as previously mentioned in Section III-C3. In our measurement system, the transmission time of a frame is recorded before the frame is passed to the AXI4-Stream clock converter at the frame generator. Before the frame is transmitted from the Ethernet port, it consumes 40 ns or more in the clock converter, 64 ns in Tri-mode Ethernet MAC (TEMAC), and 141-153 ns in Ethernet PHY, according to their datasheets [13]–[15]. Similarly, the internal delay at the receive path contains 227-235 ns in Ethernet PHY, 120 ns in TEMAC, and undocumented cycles (supposedly around 100 ns) in the AXI4-Stream data FIFO. The measured 800 ns fairly matched the theoretical one estimated from these datasheets.

We set up one of the Ethernet ports of the measurement system as a frame generator and another port as a frame capture, and connected them with different lengths of CAT6A Ethernet cable. The timestamp granularity of the frame transmit time of the frame generator and that of the receive time of the frame capture are 8 ns in the measurement system for GbE. With the NVP of 70%, the signal goes approximately 1.6 m in 8 ns.

We changed the length of the cable from 2 m to 100 m, which is long enough to see a difference in latency. To make 30 m or longer lengths, we connected multiple 10 m, 15 m, and 20 m cables by RJ45 couplers supporting CAT6A. It should be noted that, in advance, we compared the latency of a single 20 m cable with that of a 20 m joined cable composed of 5 short cables (i.e., 1 m, 1 m, 3 m, 5 m, and 10 m) with 4 couplers, and confirmed that the use of couplers does not increase latency. The influence of the couplers was not measurable at the 8 ns granularity.

The frame generator transmits 1000 frames back-to-back and the frame size was set to 1518 bytes. Figure 6 shows the averaged measured latencies of Ethernet cables. The latency of a cable proportionally increased as its length increased. For example, the 100-m cable took approximately 500 ns, which means the signal propagation speed was 200,000,000 m/s (i.e., 66.7 % of the speed of light in a vacuum). The recorded latencies of the frames were within ± 8 ns from average values, which were due to the latency deviations in the PHYs and the 8 ns granularity of the measurement system. We repeated the experiments with a frame size of 64 bytes and obtained the same results. We concluded that the measurement system accurately obtains the latency of a device under test.

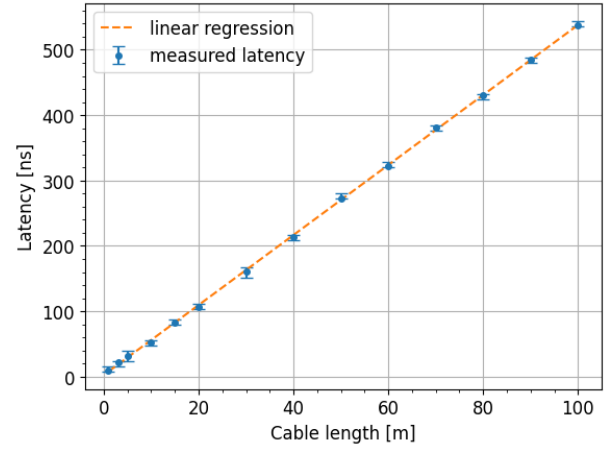


Fig. 6: Measured latencies of Ethernet cables

B. Frame capture capability

We evaluated the capability of our frame capture by receiving a sequence of frames in which frame intervals were exactly controlled. For the validation of our frame capture, we avoided using the frame generator developed by ourselves for the sake of neutrality. Instead, to transmit frames at exact reference intervals, we used an off-the-shelf network interface card (Intel i210) attached to a Linux machine. We configured a frame generator program `iperf` on the Linux machine to transmit frames of a fixed size at the line rate of GbE. Since frames are sent back-to-back, frame intervals become constant and depend on the frame size. For example, when the frame size is 64 bytes from the Ethernet header to Frame Check Sequence (FCS), the theoretical frame interval is calculated as 672 ns, considering 20 bytes of the physical layer overhead (i.e., Preamble and Interframe Gap). We connected our frame capture to the network interface card of the sender machine via an Ethernet cable and examined the frame intervals recorded by the frame capture.

We repeated the experiment with different frame sizes. No frame loss was observed for any of the tested frame sizes, and the maximum frame rate for each frame size was achieved (i.e., frames were sent back-to-back). Figure 7 shows differences between observed frame intervals by the proposed frame capture and the reference ones sent by the Linux sender machine. When reference frames are sent at a 672 ns interval, 98% of the captured frame intervals were 672 ns, and 2% were with ± 8 ns of it. As for the 12304 ns reference interval, 97% of the captured frame intervals matched it.

We confirmed that our frame capture accurately measured the received frames' arrival times, and their frame intervals matched the expected ones. A possible reason for the ± 8 ns (i.e., 1 cycle) deviations can be the jitter in the PHY IP in the FPGA board, which was previously mentioned in Section IV-A. We consider that this ± 8 ns jitter is inevitable but acceptable for expected use cases. Any end-to-end communication in Ethernet needs to go over multiple clock domains, which inevitably results in a jitter of several clock cycles in end-

TABLE I: Example of a predefined sequence of frames for synthesized frame generation

Frame index	VLAN tag	VLAN PCP	MAC SRC	MAC DST	Size	Interval (ns)
0	2349	5	0x112123	0x448111	123	372
1	128	1	0x712371	0xa37131	238	10023
2	1249	7	0x217311	0xcd1812	1389	0
3	3939	0	0xea1821	0x231028	651	1231
...	...	...	...	...	...	...
4094	633	2	0x153452	0x3c1011	1011	NA

Note that this table summarizes the key parameters to define frames in the experiments. The frame generator also allows changing other parameters such as Ethernet type, IP protocol, IP addresses, and L4 ports.

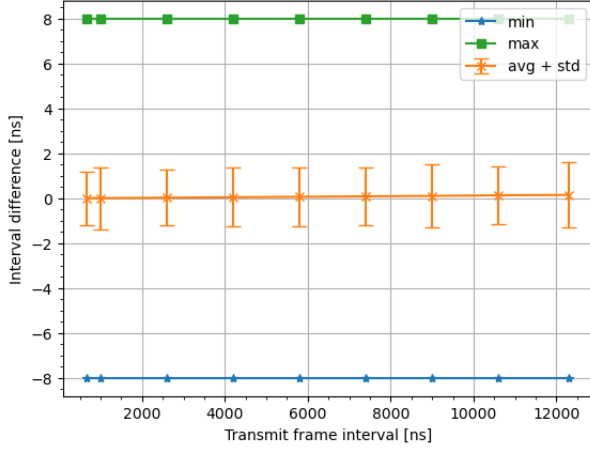


Fig. 7: Deviation of frame intervals observed by the proposed frame capture from the reference frame intervals

to-end latency. The TSN mechanisms in IEEE802.1Q-2022, which do not address this jitter, are not expected to control end-to-end latency at the granularity of 1 clock cycle. The ± 8 ns jitter of the proposed frame capture is sufficiently smaller than what needs to be measured for TSN research.

C. Frame generation capability

We conducted experiments to validate that our frame generator correctly transmits mixed frames of multiple flows from an Ethernet port according to a predefined sequence of frames. Again, for neutrality, we did not use our frame capture but Intel NIC i210, which supports hardware-based receive timestamp. It can record arrival times of frames at an 8-ns granularity [16]. We connected the frame generator to the network interface card attached to the Linux machine via an Ethernet cable.

In advance, we defined a sequence of frames to be transmitted, which was commonly used for the following tests. Each frame has the VLAN tag number randomly chosen from 0 to 4095, the priority code point (PCP) of VLAN randomly chosen from 0 to 7, and the destination and source MAC addresses randomly chosen from the predefined 256 addresses. The size of each frame was randomly selected from the shortest Ethernet frame size (i.e., 64 bytes) to the standard maximum frame size (i.e., 1522 bytes in the tagged VLAN Ethernet). The frame interval inserted before the next frame was randomly selected from 0 ns to 524,840 ns. It is worth noting that 524,840 ns is the maximum interval inserted by the

combination of the Await field of the previous frame, the NOP entry of the VLAN UDP type with the maximum payload size, and the Await field of the NOP entry.

The BRAM of the frame generator to one port can accommodate 8192 entries for its defined sequence of transmitted frames. Considering that a part of the 8192 entries was used for NOP entries, we defined 4095 frames for transmission. Please refer to Table I for an example of the defined sequence.

The frame generator transmitted 4095 frames according to the predefined sequence. On the capture side, the `tcpdump` program, with the hardware timestamp option enabled, recorded the arrival time of each frame, its frame size, and header information. We repeated the experiment and did not observe any frame loss. The headers of all the captured frames and their frame sizes exactly matched the predefined sequence in the frame generator. Results have shown that the average of the differences between a received frame interval and its predefined transmit interval was 3.0 ns, and its standard deviation was 5.0 ns. 52% of the received frame intervals exactly matched those of the sender side. 40.0% and 1.7% of the intervals were 8 ns and 16 ns (i.e., 1 cycle and 2 cycles) larger than those of the sender side, respectively. The remaining 6.3% were 8 ns smaller. By repeating the experiment, we confirmed that the average of frame intervals obtained by `tcpdump` was always approximately 3 ns larger than that of the sender side. We suspect that this is due to the possibility that the timestamp clock in the capturing network interface card was slightly slower than that of the frame generator. The frequencies of the crystal oscillators at both sides have minor deviations (e.g., 30 ppm at 25 MHz in the datasheet of the used NIC [16]). Also, the datasheet mentions possible jitter in the receiving hardware timestamp, which is 80 ns for the link speed of 100 Mbps. Although it does not present a jitter value for 1 Gbps, the observed interval differences from -8 to +16 ns were supposedly caused by jitter in the timestamp mechanism.

For comparison, we used our frame capture instead of `tcpdump`. By reproducing the same experiment, we found that 52.0% of the received intervals matched those of the sender side. 24.0% were 8 ns larger and 24.0% were 8 ns smaller. Thus, the average interval difference was 0.0 ns. We consider that the interval differences from -8 to +8 ns were due to jitter in the used PHY in the FPGA board, which is mentioned in Section IV-A.

We conclude that the frame generator correctly transmits

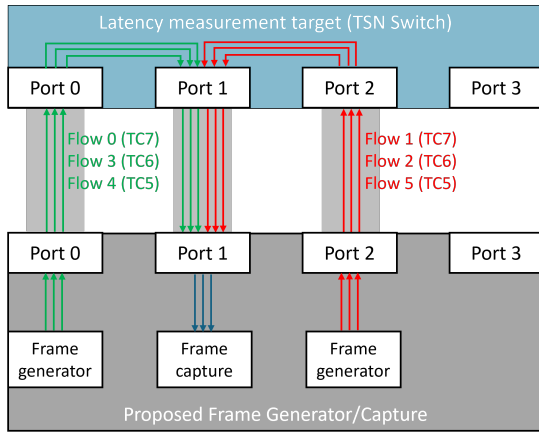


Fig. 8: The configuration of the experiment on latency bounds of TSN flows

frames with different header information, different sizes, and different intervals according to a predefined sequence, which can flexibly produce any combination of different types of flows.

D. Use case: Latency bound of multiple flows via a TSN switch

This subsection demonstrates the usefulness of our measurement system by using the proposed frame generator and the frame capture with a practical TSN switch. To do so, we adopted the *AIST-TSN* network switch that we previously proposed [17], [18] as our latency measurement target. *AIST-TSN* is an FPGA-based open-source TSN switch supporting Asynchronous Traffic Shaper (ATS) defined in the IEEE802.1Q-2022 standard [1]. We opted to use the ATS switch thanks to its advanced functionalities. ATS can ensure the worst-case latencies of network flows without time synchronization among network switches and endpoints. It also provides latency bounds for any number of priority flows. Due to the lack of a reliable measurement mechanism dedicated to TSN research, we could not confirm that our ATS implementation correctly works for complex situations where multiple flows with different characteristics contend with each other. Therefore, we consider this TSN switch to be a good candidate to demonstrate the need for a reconfigurable hardware-based frame generator and capture such as ours.

We conducted an experiment to measure the latencies of six competing different flows via the *AIST-TSN* switch, which had not been possible with the measurement system used in [17]. Figure 8 illustrates the configuration of the experiment. The frame generator connected to Port 0 of the TSN switch transmitted the frames of Flows 0, 3, and 4. The other frame generator connected to Port 2 generated Flows 1, 2, and 5. The switch forwarded all the flows to Port 1. The frame capture connected to Port 1 recorded the arrival times of all the frames.

The frames of Flow 0 contained the priority number in its Priority Code Point (PCP) field of the VLAN header, which was mapped to Traffic Class 7 (i.e., the highest priority) in the TSN switch. The size of each frame of Flow 0 was 1542

bytes, including the 20-byte overheads at the physical layer (i.e., Preamble and Interframe Gap). The transmission rate was 100 Mbps and the burst size was 3084 bytes. In the TSN switch, ATS was assigned for the transmission selection mechanism of Traffic Class 7. The Committed Information Rate (CIR) of the ATS scheduler of Flow 0 was set to the same rate as the incoming rate, and the Committed Burst Size (CBS) was also set to the same burst size. Flow 3 was mapped to the second-highest priority (i.e., Traffic Class 6). The burst size was 1542 bytes. Flow 4 was mapped to the lowest priority with the transmission selection mechanism of Strict Priority (SP), which was treated as best-effort traffic. The maximum burst size in the flow was 13878 bytes. Table II summarizes the parameters used for all the transmitted flows in this experiment.

The theoretical model of ATS allows us to calculate for each flow the upper bound of the latency increase caused by the contention at the egress port of the switch (i.e., Port 1 of the TSN switch in this experiment). In [17], authors represented the upper-bound latency increase in the k -th network switch for a given flow f as $d_{CO,max}(k, f)$. However, they only assumed that only one ATS flow exists in a traffic class. As discussed in [19], if multiple ATS flows exist in a traffic class, it is necessary to extend it to $\max_{g \in F_s(k, f)} d_{CO,max}(k, g)$, which takes the maximum of the contention delays of all the ATS flows in the same traffic class, $F_s(k, f)$. Please refer to Table II on the second row to see the calculated (theoretical) upper bound of the contention delay of each ATS flow.

The obtained measured results are also summarized in the third row in Table II. The ATS flows did not involve any frame loss. The SP flows involved 60% frame losses, and their observed latencies were only of the successfully received frames. The maximum latencies of Flow 0 and Flow 1 (i.e., the ATS flows of the highest priority) were approximately 76.2 us and 74.8 us, respectively. Their actual contention delays were calculated as the differences between their maximum and minimum latencies (represented by “Maximum - Minimum” in Table II), which were approximately 24.6 and 23.2 us. These contention delays were successfully less than their theoretical upper bound (i.e., 37.0 us). Similarly, the actual contention delays of Flow 2 and Flow 3 (i.e., the ATS flows of the second highest priority) were approximately 47.7 us and 48.4 us, which were below their theoretical bound (i.e., 92.5 us).

Note that the theoretical model of the contention delay does not estimate the maximum delay that is expected to be observed. Instead, it assures that the observed latencies of the frames of the flow never exceed the calculated upper bound. The model can calculate the upper bounds on contention delays in any complex situations, but it tends to output values larger than those actually observed. Flow 4 and Flow 5 experienced approximately 60% of frame losses, and their observed latencies varied from 13.7 us to 379.4 us. Since the TSN switch prioritizes the frames of the ATS flows, the frames of the SP flows waited a long time to be transmitted, and eventually, some of them overflowed their queues. Because the components used for SP flows in the switch are much

TABLE II: The parameters of flows, their theoretical contention delay bounds and observed latencies

		Flow 0	Flow 1	Flow 2	Flow 3	Flow 4	Flow 5
Transmitted flows' parameters	Traffic class	7 (ATS)	7 (ATS)	6 (ATS)	6 (ATS)	5 (SP)	5 (SP)
	Input port of the switch	0	2	2	0	0	2
	Frame size (byte)	1542	1542	1542	1542	1542	1542
	Transmit rate (Mbps)	100	100	100	100	800	800
	Burst size (byte)	3084	1542	3084	1542	13878	13878
Theoretical ATS contention delay bound (us)	$\max_{g \in F_s(k,f)} d_{CO,max}(k,g)$	37.008	37.008	92.520	92.520	N.A.	N.A.
Measured latency (us)	Averaged	59.014	58.436	63.697	61.934	315.508	316.185
	Minimum	51.648	51.640	51.640	51.640	100.064	13.736
	Maximum	76.216	74.808	99.384	100.064	379.416	379.152
	Maximum - Minimum	24.568	23.168	47.744	48.424	279.352	365.416

In the TSN switch, the CIR and CBS of the ATS scheduler for each ATS flow were set to the same values as its transmit rate and burst size at the frame generators.

TABLE III: Proposed EFCC system hardware complexity

Component	MAC	FIFO	Frame Gen.	AXI4 Stream Switch	Timestamp Rec.	Clk. Conv.	Others	Total	Utiliz. (%)
LUTs	8299	2796	19148	594	22735	265	7853	61690	30.27
FFs	13492	1992	27959	1163	33400	448	12771	91225	22.38
LUTRAMs	1357	2048	3875	0	5152	32	2198	14662	22.91
BRAMs	0	0	152	0	256	0	2	410	92.25

simpler than those of ATS, the first frame of Flow 5, which was processed by the switch before congestion began earnestly, resulted in short latency (i.e., 13.7 us).

E. Hardware resource utilization

In this evaluation, we present the hardware complexity of the proposed EFCC when implemented on an AMD Xilinx Kintex 7 FPGA KC705 Evaluation Kit [11] using the AMD Xilinx Vivado 2022.1 tool [20]. Table III showcases the resource utilization of each component of our system. The synthesized configuration is that of the one depicted in Figure 1, with the shared buffer configuration illustrated in Figure 4b. In other words, the “Frame Gen.” column represents four instances of the FG while the “Timestamp Rec.” shows that of eight instances of the TR.

It can be seen from this table that the entire proposed switch consumed a reasonable portion of the LUTs, FFs, and LUTRAMs. However, in terms of BRAM blocks, the design is almost reaching its limit. As depicted in this table, the two main components of our system, the FG and TR, consume most of the available BRAM resources. These two modules contain several buffers where the transmission information and the recorded timestamps are stored, respectively. Nonetheless, the current implementation supports four frame generators and eight in-line timestamp recorders, enabling the full exploitation of the BRAM resources of the KC705 board. Moreover, as previously mentioned, instead of the standalone mode, the proposed system can be easily attached to a host machine whenever much larger frame-generation/capture buffers are needed. Therefore, we believe that the obtained benefits with the current configuration within the utilized resources are very practical. Furthermore, the used KC705 evaluation board is among the most affordable and low-power FPGA development boards, featuring relatively limited resources compared to other mid- or top-tier commercially available counterparts. We assert that the current architecture enables the implementation

of additional TSN measurement features in larger boards while maintaining FPGA costs at a minimum.

V. CONCLUSION

In this paper, we propose an FPGA-based Ethernet Frame Crafter & Capture (EFCC) system targeted for TSN measurement and research. EFCC supports the generation of multiple TSN flows where different frame sizes, frame rates, and burst sizes can be independently set for each flow. It can capture the arrival times of frames with high precision without any loss. EFCC was implemented on an affordable FPGA in a reconfigurable manner to allow the measurement of TSN systems under different traffic scenarios. Yet, it can be ported to other FPGA boards with minimum effort.

The evaluation results confirmed that our proposed measurement mechanism is useful for analyzing detailed behaviors of TSN mechanisms. In fact, the proposed frame generator allows the mixing of multiple flows of various characteristics. In addition, the proposed frame capture is capable of recording the transmit and receive timestamps of frames in an 8-ns/6.4-ns precision. The proposed mechanism has shown its ability to be programmable and accurate, which we believe is indispensable for the development of realistic TSN technologies.

We plan to validate TSN switches in more complex situations where multiple and various flows with different sizes and parameters coexist in a given TSN network. In addition, the proposed design should be validated under higher line-rates to accommodate a broader spectrum of TSN applications.

ACKNOWLEDGMENT

This paper is based on results obtained from the project, “Research and Development Project of the Enhanced infrastructures for Post-5G Information and Communication Systems” (JPNP20017), commissioned by the New Energy and Industrial Technology Development Organization (NEDO).

REFERENCES

- [1] *IEEE Standard for Local and Metropolitan Area Networks—Bridges and Bridged Networks*, Institute of Electrical and Electronics Engineers, 2022.
- [2] A. Nasrallah, A. Thyagaturu, Z. Alharbi, C. Wang, X. Shao, M. Reisslein, and H. Elbakoury, “Performance comparison of iee 802.1 tsn time aware shaper (tas) and asynchronous traffic shaper (ats),” *IEEE Access*, vol. 7, pp. 44 165–44 181, 2019.
- [3] *The AIST-TSN project repository*, <https://github.com/CCIRT/aist-tsn/>, 2025.
- [4] R. Debnath, P. Hortig, L. Zhao, and S. Steinhurst, “Advanced modeling and analysis of individual and combined tsn shapers in omnet++,” in *2023 IEEE 29th International Conference on Embedded and Real-Time Computing Systems and Applications (RTCSA)*, 2023, pp. 176–185.
- [5] J. Jiang, Y. Li, S. H. Hong, A. Xu, and K. Wang, “A time-sensitive networking (tsn) simulation model based on omnet++,” in *2018 IEEE International Conference on Mechatronics and Automation (ICMA)*, 2018, pp. 643–648.
- [6] J. Falk, D. Hellmanns, B. Carabelli, N. Nayak, F. Dürr, S. Kehrer, and K. Rothermel, “Nesting: Simulating iee time-sensitive networking (tsn) in omnet++,” in *2019 International Conference on Networked Systems (NetSys)*, 2019, pp. 1–8.
- [7] F. Costa, F. Vogt, F. Rodriguez, M. Luizelli, and C. Rothenberg, “Towards time-sensitive networking traffic generation with pipo-tg,” in *Anais Estendidos do XLII Simpósio Brasileiro de Redes de Computadores e Sistemas Distribuídos*. Porto Alegre, RS, Brasil: SBC, 2024, pp. 73–80.
- [8] C. Li, T. Li, J. Li, W. Fu, and B. Wang, “Dra: Ultra-low latency network i/o for tsn embedded end-systems,” in *2023 IEEE/ACM 31st International Symposium on Quality of Service (IWQoS)*, 2023, pp. 01–10.
- [9] C. Li, Z. Li, T. Li, C. Li, and B. Wang, “A deterministic embedded end-system tightly coupled with tsn schedule,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 42, no. 11, pp. 3707–3719, 2023.
- [10] F. G. Costa, F. G. Vogt, F. R. Cesen, A. G. de Castro, M. C. Luizelli, and C. E. Rothenberg, “Pipo-tg: Parameterizable high-performance traffic generation,” in *NOMS 2024-2024 IEEE Network Operations and Management Symposium*, 2024, pp. 1–9.
- [11] AMD Xilinx Cooperation, *KC705 Evaluation Board for the Kintex-7 FPGA, UG810 (v1.9)*, 2019.
- [12] O. E. D. Inc., <https://ethermetfmc.com/>, 2022.
- [13] Advanced Micro Devices, Inc., *AXI4-Stream Infrastructure IP Suite v3.0 LogiCORE IP Product Guide (PG085)*, v3.0, 2023.
- [14] Marvell Technology, *Marvell Alaska 88E1510/88E1518/88E1512/88E1514, Integrated 10/100/1000 Mbps Energy Efficient Ethernet Transceiver*, 2022.
- [15] Advanced Micro Devices, Inc., *Tri-Mode Ethernet MAC LogiCORE IP Product Guide (PG051)*, v9.0, 2023.
- [16] Intel Cooperation, *Intel Ethernet Controller I210 Datasheet, Revision Number: 3.7*, 2021.
- [17] A. Ben Ahmed, T. Hirofuchi, and T. Fukai, “Hardware design and evaluation of an fpga-based network switch supporting asynchronous traffic shaping for time sensitive networking,” *IEEE Access*, vol. 12, pp. 123 149–123 165, 2024.
- [18] —, “Fpga-based network switch architecture supporting credit based shaper for time sensitive networks,” in *Proceedings of the 29th IEEE International Conference on Emerging Technologies and Factory Automation - ETFA2024*, 2024, pp. 1–8.
- [19] J. Specht and S. Samii, “Urgency-based scheduler for time-sensitive switched ethernet networks,” in *2016 28th Euromicro Conference on Real-Time Systems (ECRTS)*, 2016, pp. 75–85.
- [20] AMD Xilinx Cooperation, *Vivado Design Suite User Guide: Getting Started (UG910)*, 2022.